

1. 掛載雲端硬碟

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

2. pip install scikit-optimize

```
!pip install scikit-optimize
```

```
Collecting scikit-optimize
  Downloading scikit_optimize-0.10.2-py2.py3-none-any.whl.metadata (9.7 kB)
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.11/dist-packages (from scikit-optimize) (1.4.2)
Collecting pyaml>=16.9 (from scikit-optimize)
  Downloading pyaml-25.1.0-py3-none-any.whl.metadata (12 kB)
Requirement already satisfied: numpy>=1.20.3 in /usr/local/lib/python3.11/dist-packages (from scikit-optimize) (2.0.2)
Requirement already satisfied: scipy>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-optimize) (1.14.1)
Requirement already satisfied: scikit-learn>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from scikit-optimize) (1.6.1)
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.11/dist-packages (from scikit-optimize) (24.2)
Requirement already satisfied: PyYAML in /usr/local/lib/python3.11/dist-packages (from pyaml>=16.9->scikit-optimize) (6.0.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn>=1.0.0->scikit-optimize) (3.6.0)
Downloading scikit_optimize-0.10.2-py2.py3-none-any.whl (107 kB)
----- 107.8/107.8 kB 5.6 MB/s eta 0:00:00

Downloading pyaml-25.1.0-py3-none-any.whl (26 kB)
Installing collected packages: pyaml, scikit-optimize
Successfully installed pyaml-25.1.0 scikit-optimize-0.10.2
```

```
!pip uninstall -y numpy
!pip uninstall -y catboost
!pip install numpy==2.0 # You can specify a specific version if needed
!pip install catboost
```

```
Found existing installation: numpy 1.23.5
Uninstalling numpy-1.23.5:
  Successfully uninstalled numpy-1.23.5
WARNING: Skipping catboost as it is not installed.
Collecting numpy==2.0
  Downloading numpy-2.0.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (60 kB)
----- 60.9/60.9 kB 1.8 MB/s eta 0:00:00

Downloading numpy-2.0.0-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (19.3 MB)
----- 19.3/19.3 MB 30.0 MB/s eta 0:00:00

Installing collected packages: numpy
Successfully installed numpy-2.0.0
Collecting catboost
  Using cached catboost-1.2.8-cp311-cp311-manylinux2014_x86_64.whl.metadata (1.2 kB)
Requirement already satisfied: graphviz in /usr/local/lib/python3.11/dist-packages (from catboost) (0.20.3)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (from catboost) (3.10.0)
Requirement already satisfied: numpy<3.0,>=1.16.0 in /usr/local/lib/python3.11/dist-packages (from catboost) (2.0.0)
Requirement already satisfied: pandas>=0.24 in /usr/local/lib/python3.11/dist-packages (from catboost) (2.2.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.11/dist-packages (from catboost) (1.14.1)
Requirement already satisfied: plotly in /usr/local/lib/python3.11/dist-packages (from catboost) (5.24.1)
Requirement already satisfied: six in /usr/local/lib/python3.11/dist-packages (from catboost) (1.17.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas>=0.24->catboost) (2025.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (4.57.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (24.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib->catboost) (3.2.3)
Requirement already satisfied: tenacity>=6.2.0 in /usr/local/lib/python3.11/dist-packages (from plotly->catboost) (9.1.2)
Using cached catboost-1.2.8-cp311-cp311-manylinux2014_x86_64.whl (99.2 MB)
Installing collected packages: catboost
Successfully installed catboost-1.2.8
```

```
# ----- 套件載入 -----
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import skew
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import RobustScaler
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from sklearn.ensemble import VotingClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_predict, cross_val_score
from sklearn.metrics import roc_auc_score, roc_curve, confusion_matrix, classification_report, ConfusionMatrixDisplay
from sklearn.preprocessing import RobustScaler
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
```

```

from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, LabelEncoder
from sklearn.base import BaseEstimator, TransformerMixin
from skopt import BayesSearchCV
from skopt.space import Real, Integer
import matplotlib.pyplot as plt
import shap
import time
from tqdm.notebook import tqdm
import shap
import json
import warnings
warnings.filterwarnings('ignore')

```

```

# ----- 載入資料 -----
train = pd.read_csv('/content/drive/MyDrive/Data_Mining/train.csv')
test = pd.read_csv('/content/drive/MyDrive/Data_Mining/test.csv')

# ----- 初步資料探索 (EDA) -----
print("Train head:")
train_df = pd.DataFrame(train)
print(train_df.head())

print("\nTrain describe:")
test_df = pd.DataFrame(test)
print(test_df.head())

```

```

Train head:
   id  age  height(cm)  weight(kg)  waist(cm)  eyesight(left) \
0  0  25.0      170.0      60.0      75.0      1.5
1  1  40.0      165.0      65.0      79.0      1.0
2  2  50.0      170.0      80.0      93.0      1.2
3  3  75.0      160.0      60.0      89.0      0.8
4  4  60.0      170.0      75.0      89.0      0.8

   eyesight(right)  hearing(left)  hearing(right)  systolic  ...  HDL  LDL \
0                1.2            1.0            1.0    100.0  ...  50.0  81.0
1                1.2            1.0            1.0    130.0  ...  58.0  130.0
2                1.0            1.0            1.0    130.0  ...  44.0  130.0
3                1.0            1.0            1.0    116.0  ...  44.0  149.0
4                0.8            1.0            1.0    138.0  ...  38.0  122.0

   hemoglobin  Urine protein  serum creatinine  AST  ALT  Gtp \
0         15.2            1.0                0.8  18.0  16.0  19.0
1         12.8            1.0                0.6  31.0  26.0  33.0
2         16.7            1.0                0.9  38.0  46.0  69.0
3         15.1            1.0                1.1  24.0  22.0  20.0
4         14.6            1.0                1.0  26.0  27.0  97.0

   dental caries  smoking
0              1.0      1.0
1              0.0      0.0
2              0.0      1.0
3              0.0      0.0
4              0.0      1.0

[5 rows x 24 columns]

```

```

Train describe:
   id  smoking
0  15000  0.913770
1  15001  0.012991
2  15002  0.025699
3  15003  0.708826
4  15004  0.340198

```

```

# ----- 特徵工程：視力、聽力欄位處理 -----
def create_extra_features(df):
    # 視力超過 9 的視為異常值，修正為 0.0 (極差視力)

    # 聽力也是相同邏輯：0表示正常，數字越大越差
    best_hearing = np.minimum(df['hearing(left)'], df['hearing(right)'])
    worst_hearing = np.maximum(df['hearing(left)'], df['hearing(right)'])
    df['hearing(left)'] = best_hearing - 1
    df['hearing(right)'] = worst_hearing - 1

create_extra_features(train)
create_extra_features(test)

```

```

-----
KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.11/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3804         try:
-> 3805             return self._engine.get_loc(casted_key)
    3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'hearing(left)'

```

The above exception was the direct cause of the following exception:

```

KeyError                                Traceback (most recent call last)
<ipython-input-8-1c216e1d814f> in <cell line: 0>()
     11
     12 create_extra_features(train)
--> 13 create_extra_features(test)

<ipython-input-8-1c216e1d814f> in create_extra_features(df)
      5
      6     # 聽力也是相同邏輯：0表示正常，數字越大越差
----> 7     best_hearing = np.minimum(df['hearing(left)'], df['hearing(right)'])
      8     worst_hearing = np.maximum(df['hearing(left)'], df['hearing(right)'])
      9     df['hearing(left)'] = best_hearing - 1

----- 1 frames -----
/usr/local/lib/python3.11/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3810         ):
    3811             raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
    3813         except TypeError:
    3814             # If we have a listlike key, _check_indexing_error will raise

KeyError: 'hearing(left)'

```

```

# ----- 特徵工程：衍生欄位 -----
def add_derived_features(df):
    df['BMI'] = df['weight(kg)'] / (df['height(cm)'] / 100) ** 2
    df['BP_ratio'] = df['systolic'] / df['relaxation']
    df['liver_ratio'] = df['ALT'] / df['AST']
    df['cholesterol_ratio'] = df['LDL'] / df['HDL']

add_derived_features(train)
add_derived_features(test)

```

```

# 取得所有數值型特徵（排除 'id' 和 'smoking'）
numeric_features = train.select_dtypes(include=['float64', 'int64']).drop(columns=['id', 'smoking']).columns

# 計算數值特徵的數量
num_features = len(numeric_features)

# 根據特徵數量設置網格大小
ncols = 5 # 每行顯示 5 張圖
nrows = (num_features // ncols) + (num_features % ncols > 0) # 根據特徵數量計算需要的行數

# 設定畫布大小：一個直方圖圖形
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(20, nrows * 4))
axes = axes.flatten() # 轉換為一維陣列，方便迭代

# 計算偏度並繪製直方圖
for i, col in enumerate(numeric_features):
    # 繪製直方圖
    sns.histplot(train[col], bins=30, kde=True, ax=axes[i])

    # 計算偏度
    skewness_value = skew(train[col].dropna()) # 去除 NaN 值以避免錯誤

    # 在圖上添加偏度註記
    axes[i].text(0.95, 0.95, f'Skewness: {skewness_value:.2f}',
                transform=axes[i].transAxes,
                fontsize=10,
                verticalalignment='top',
                horizontalalignment='right',
                bbox=dict(boxstyle="round,pad=0.3", edgecolor="black", facecolor="white"))

# 設置標題和軸標籤
axes[i].set_title(f'Distribution of {col}')
axes[i].set_xlabel('')
axes[i].set_ylabel('')

```

```
# 隱藏多餘的子圖
for j in range(i + 1, len(axes)):
    axes[j].axis('off')

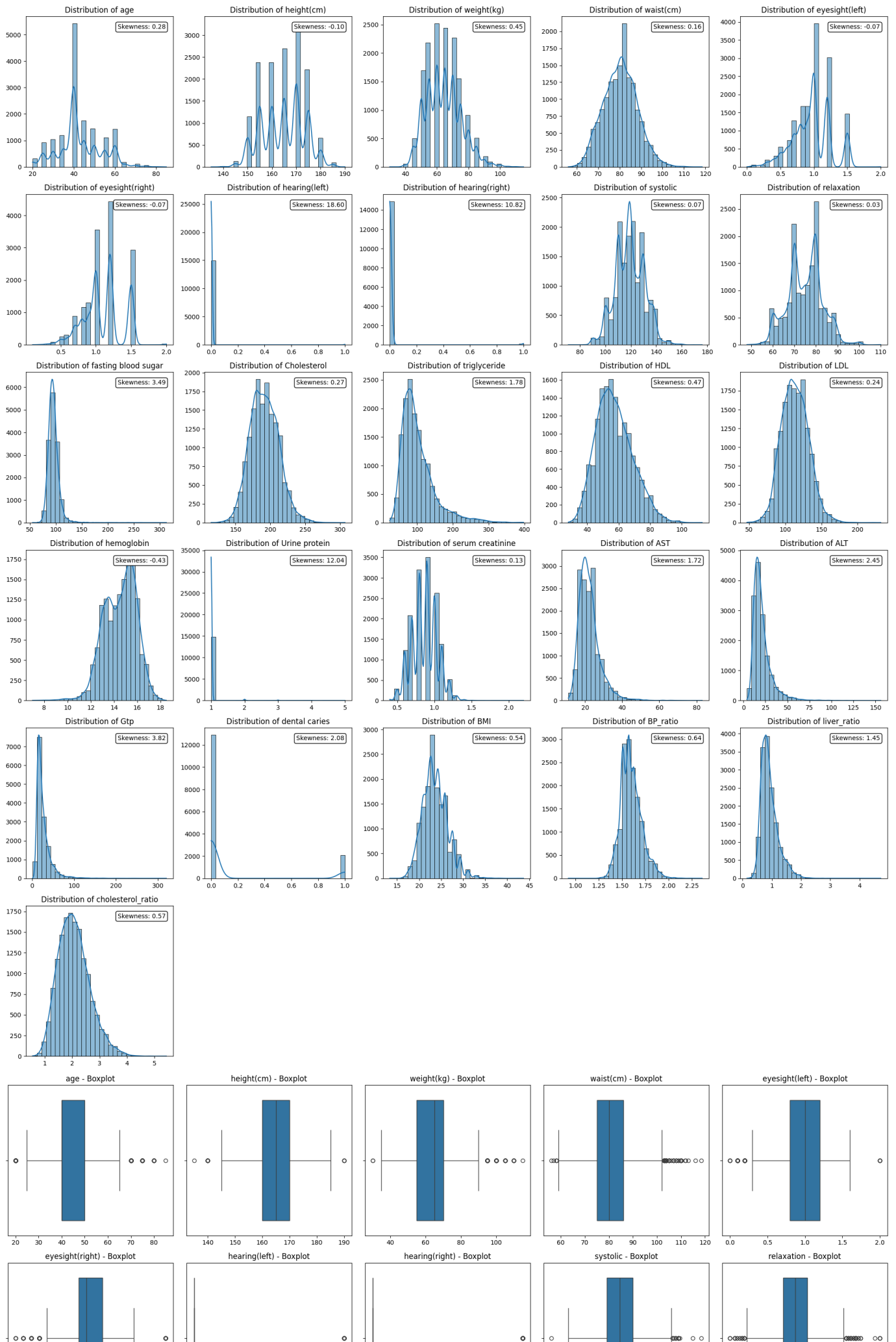
# 調整子圖間距
plt.tight_layout()
plt.show()

# 設定畫布大小：箱型圖
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(20, nrows * 4))
axes = axes.flatten() # 轉換為一維陣列，方便迭代

# 依序繪製每個特徵的箱型圖
for i, col in enumerate(numeric_features):
    sns.boxplot(x=train[col], ax=axes[i])
    axes[i].set_title(f'{col} - Boxplot')
    axes[i].set_xlabel('')

# 隱藏多餘的子圖
for j in range(i + 1, len(axes)):
    axes[j].axis('off')

# 調整子圖間距
plt.tight_layout()
plt.show()
```

```
# ----- 分離目標與特徵 -----
X_train = train.drop(columns=['smoking', 'id'])
y_train = train['smoking']
X_test = test.drop(columns=['id'])
```

```
# ----- 分離類別特徵與連續特徵 -----
to_ohe = ['hearing(left)', 'hearing(right)', 'dental caries']
X_train_ohe = X_train[to_ohe]
X_test_ohe = X_test[to_ohe]

# 在這裡，我們將 age 也視為連續特徵
X_train_cont = X_train.drop(columns=to_ohe + ['Urine protein']) # 移除類別特徵與 age
X_test_cont = X_test.drop(columns=to_ohe + ['Urine protein'])

# ----- 定義 frequency encoding 函式 -----
def frequency_encode(train_col, test_col):
    """用訓練資料做 freq 編碼，回傳轉換後的 train, test 欄位"""
    freq_map = train_col.value_counts(normalize=True) # 計算比例 (可選擇 normalize=False 用出現次數)
    return train_col.map(freq_map), test_col.map(freq_map).fillna(0)

# ----- 對 age 做 freq encoding -----
X_train_cont['age_freq'], X_test_cont['age_freq'] = frequency_encode(X_train_cont['age'], X_test_cont['age'])

# 現在 X_train_cont 和 X_test_cont 都包含 age 和 age_freq 特徵
```

```
Gtn - Boxplot      dental caries - Boxplot      BMI - Boxplot      BP_ratio - Boxplot      liver_ratio - Boxplot

# ----- 偏態處理 loglp -----
numeric_cols = X_train_cont.select_dtypes(include=['float64', 'int64']).columns.drop(['age_freq'])

# 找出偏態大於 1 的欄位做 loglp
skewed_features = [col for col in numeric_cols if skew(X_train_cont[col]) > 1]
import math

# 設置每行兩張圖（左為原始，右為 loglp 轉換）
ncols = 2
nrows = math.ceil(len(skewed_features)) # 每行一特徵，但兩張圖
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(12, nrows * 4))

log_transformed_features = [] # 記錄轉換過的特徵

# 繪製每個特徵的直方圖
for i, col in enumerate(skewed_features):
    # 繪製原始特徵直方圖（左側圖）
    sns.histplot(X_train_cont[col], bins=30, kde=True, ax=axes[i, 0])
    skewness_before = skew(X_train_cont[col].dropna())
    axes[i, 0].set_title(f'{col} - Before loglp\nSkewness: {skewness_before:.2f}')
    axes[i, 0].set_xlabel('')
    axes[i, 0].set_ylabel('')

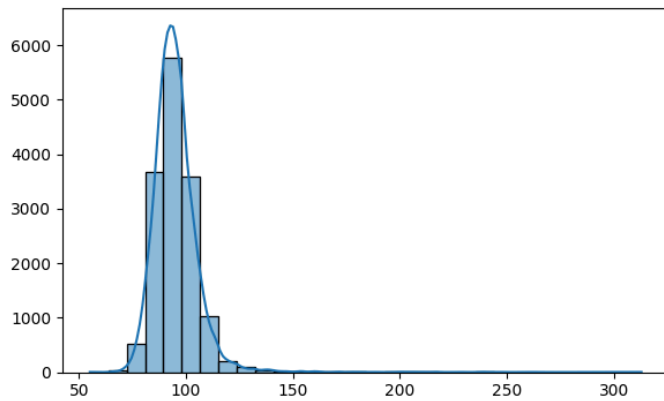
    # 確認數值非負並執行 loglp 轉換
    if (X_train_cont[col] >= 0).all() and (X_test_cont[col] >= 0).all():
        X_train_cont[col] = np.log1p(X_train_cont[col])
        X_test_cont[col] = np.log1p(X_test_cont[col])
        log_transformed_features.append(col)

    # 繪製轉換後特徵直方圖（右側圖）
    sns.histplot(X_train_cont[col], bins=30, kde=True, ax=axes[i, 1])
    skewness_after = skew(X_train_cont[col].dropna())
    axes[i, 1].set_title(f'{col} - After loglp\nSkewness: {skewness_after:.2f}')
    axes[i, 1].set_xlabel('')
    axes[i, 1].set_ylabel('')

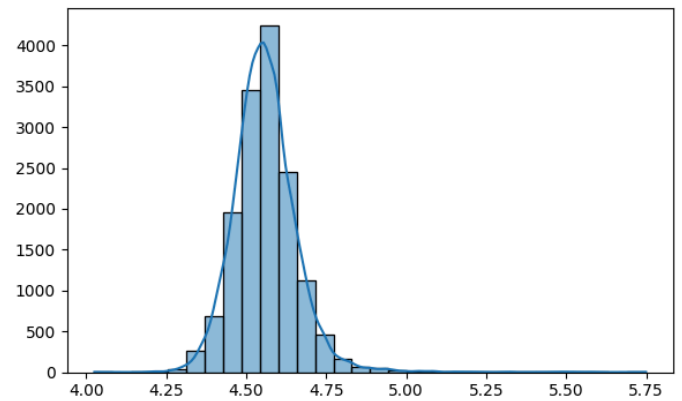
# 調整子圖布局
plt.tight_layout()
plt.show()

# 打印已轉換的特徵
print("Log-transformed features:", log_transformed_features)
```

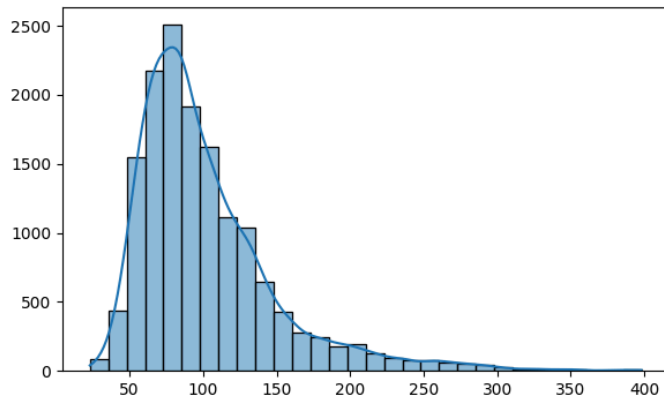

fasting blood sugar - Before log1p
Skewness: 3.49



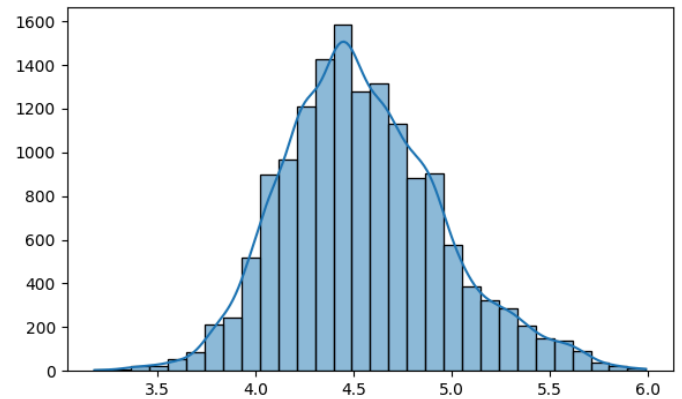
fasting blood sugar - After log1p
Skewness: 1.29



triglyceride - Before log1p
Skewness: 1.78



triglyceride - After log1p
Skewness: 0.40



AST - Before log1p
Skewness: 1.72



AST - After log1p
Skewness: 0.57



```
# ----- 標準化: RobustScaler 對抗極端值 -----
sc = RobustScaler()
X_train_scaled = pd.DataFrame(sc.fit_transform(X_train_cont.drop(columns=['age_freq'])), columns=X_train_cont.drop(columns=['age_freq']).columns)
X_test_scaled = pd.DataFrame(sc.transform(X_test_cont.drop(columns=['age_freq'])), columns=X_test_cont.drop(columns=['age_freq']).columns)

# 加回 age_freq 特徵
X_train_scaled['age_freq'] = X_train_cont['age_freq']
X_test_scaled['age_freq'] = X_test_cont['age_freq']
```

```
# ----- 分離類別特徵與數值特徵 -----
to_oh = ['hearing(left)', 'hearing(right)', 'dental caries']
X_train_oh = X_train[to_oh]
X_test_oh = X_test[to_oh]

# 對 'Urine protein' 使用 Label Encoding
le = LabelEncoder()
X_train['Urine protein'] = le.fit_transform(X_train['Urine protein'])
X_test['Urine protein'] = le.transform(X_test['Urine protein'])

# ----- One-Hot Encoding 對其他類別特徵進行 One-Hot Encoding -----
X_train_oh = pd.get_dummies(X_train_oh, columns=to_oh)
X_test_oh = pd.get_dummies(X_test_oh, columns=to_oh)

# ----- 合併數值欄位與已處理的類別欄位 -----
X_train_final = pd.concat([X_train_scaled, X_train_oh, X_train[['Urine protein']], axis=1)
X_test_final = pd.concat([X_test_scaled, X_test_oh, X_test[['Urine protein']], axis=1)

# 確保測試集與訓練集的特徵一致
X_test_final = X_test_final.reindex(columns=X_train_final.columns, fill_value=0)
```

Gcp - Before log1p

Gcp - After log1p

```
# ----- 過濾高相關特徵 (optional, 可略過) -----
# 可視需求停用以下區塊

# 計算相關性矩陣
corr_matrix = X_train_final.corr().abs()

# 找出相關性大於 0.8 的特徵並準備過濾
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
to_drop = [col for col in upper.columns if any(upper[col] > 0.8)]

# 繪製相關性熱圖
plt.figure(figsize=(12, 10)) # 設定畫布尺寸
```

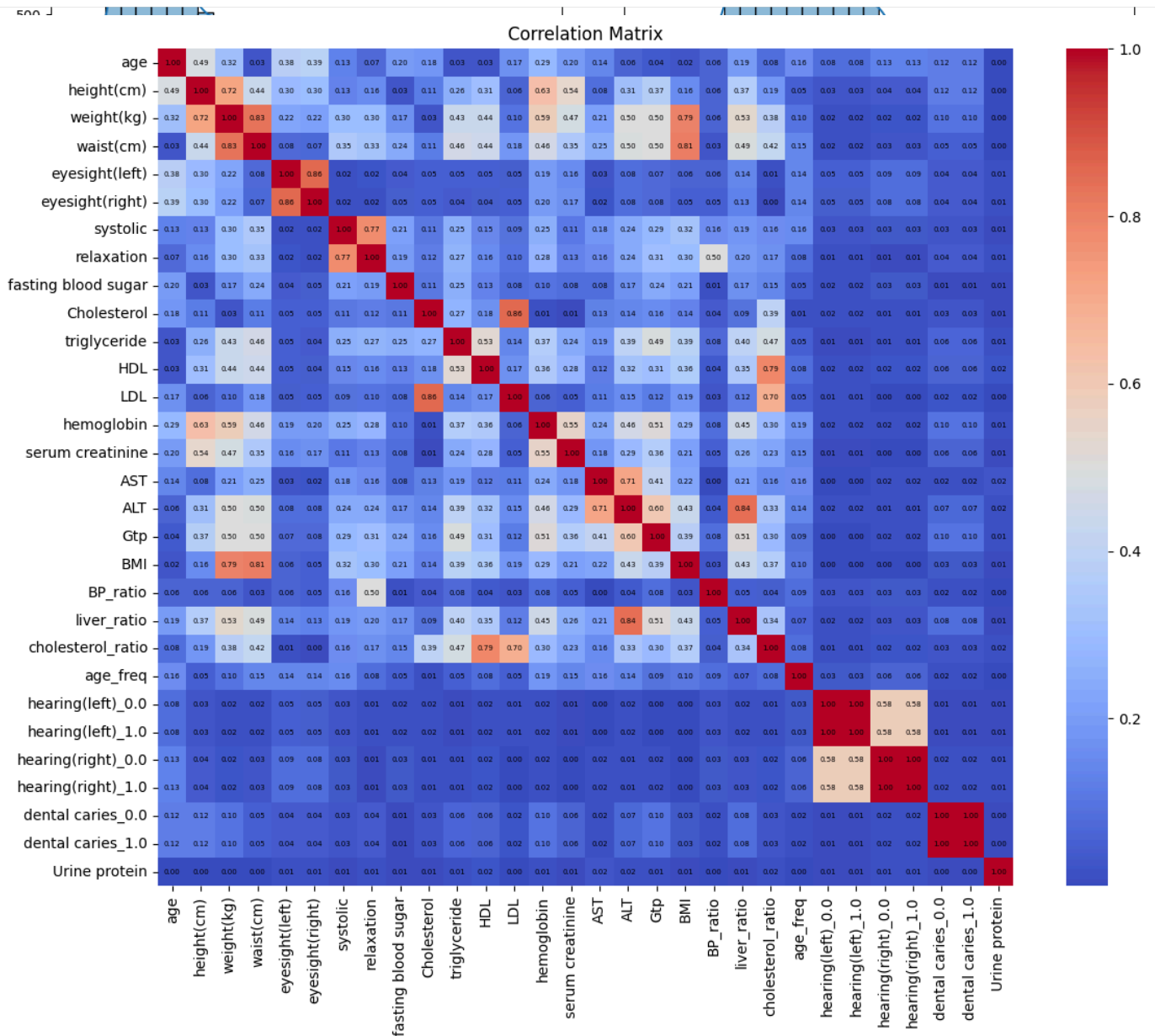
```

sns.heatmap(
    corr_matrix,
    annot=True,      # 顯示數值
    fmt=".2f",       # 數值格式
    cmap='coolwarm',
    cbar=True,
    annot_kws={"fontsize": 5, "color": "black"} # 調整文字大小和顏色
)
plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()

# 回報被過濾的特徵
print("Features dropped due to high correlation (>0.8):", to_drop)

X_train_final = X_train_final.drop(columns=to_drop)
X_test_final = X_test_final.drop(columns=to_drop)

```



Features dropped due to high correlation (>0.8): ['waist(cm)', 'eyesight(right)', 'LDL', 'BMI', 'liver_ratio', 'hearing(left)_1.0', 'hearing(right)_1.0', 'dental caries_1.0', 'Urine protein']

```

# ----- 自訂進度條 callback -----
class TqdmCallback:
    def __init__(self, total_iter):
        self.pbar = tqdm(total=total_iter)

    def __call__(self, res):
        self.pbar.update(1)
        if self.pbar.n % 5 == 0:
            print(f"[{self.pbar.n}/{self.pbar.total}] 進度更新")

# ----- 模型評估圖 -----
def plot_model_evaluation(opt, X_train, y_train, cv, title_prefix="Model"):
    auc_scores = opt.cv_results_['mean_test_score']

```

```

plt.figure(figsize=(14, 5))
plt.subplot(1, 2, 1)
plt.plot(range(1, len(auc_scores)+1), auc_scores, marker='o', label='AUC Score')
plt.axhline(y=max(auc_scores), color='r', linestyle='--', label=f"Best AUC = {max(auc_scores):.4f}")
plt.xlabel("Iteration")
plt.ylabel("Mean AUC (CV)")
plt.title(f"{title_prefix} AUC Learning Curve")
plt.legend()
plt.grid(True)

y_pred_proba_cv = cross_val_predict(opt.best_estimator_, X_train, y_train, cv=cv, method='predict_proba')[:, 1]
fpr, tpr, _ = roc_curve(y_train, y_pred_proba_cv)
auc_score = roc_auc_score(y_train, y_pred_proba_cv)

plt.subplot(1, 2, 2)
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {auc_score:.4f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f"{title_prefix} ROC Curve")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# ----- SHAP 選擇重要特徵 -----
def shap_feature_selection(model, X_train, top_n=30):
    explainer = shap.TreeExplainer(model)
    shap_values = explainer.shap_values(X_train)
    mean_abs_shap = np.abs(shap_values).mean(axis=0)
    shap_importance = pd.DataFrame({
        'feature': X_train.columns,
        'importance': mean_abs_shap
    }).sort_values(by='importance', ascending=False)

    shap.summary_plot(shap_values, X_train, max_display=top_n, plot_type="bar")
    return shap_importance.head(top_n)['feature'].tolist()

# ----- 主流程 -----
def main(X_train_final, X_test_final, y_train, test_id):
    # 定義模型與搜尋空間
    model = XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='logloss')
    param_space = {
        'learning_rate': Real(0.01, 0.3, prior='log-uniform'),
        'n_estimators': Integer(100, 500),
        'max_depth': Integer(3, 10),
        'subsample': Real(0.5, 1.0),
        'colsample_bytree': Real(0.5, 1.0),
        'min_child_weight': Integer(1, 10),
        'gamma': Real(0, 1.0),
        'reg_alpha': Real(0, 1.0),
        'reg_lambda': Real(0, 1.0),
        'scale_pos_weight': Real(1, 10)
    }

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    opt = BayesSearchCV(
        model,
        search_spaces=param_space,
        cv=cv,
        n_iter=70,
        scoring='roc_auc',
        n_jobs=-1,
        random_state=42,
        verbose=0
    )

    print("🚧 開始 Bayesian Optimization...")
    start = time.time()
    opt.fit(X_train_final, y_train, callback=TqdmCallback(70))
    print(f"✅ 搜尋完成, 耗時: {time.time() - start:.2f} 秒")

    print("最佳參數: ", opt.best_params_)
    print(f"最佳 AUC: {opt.best_score_:.4f}")

    # ----- 特徵重要性 -----
    plt.figure(figsize=(10, 6))
    sns.barplot(x=opt.best_estimator_.feature_importances_, y=X_train_final.columns)
    plt.title("Feature Importances")
    plt.tight_layout()
    plt.show()

    plot_model_evaluation(opt, X_train_final, y_train, cv, title_prefix="XGBoost")

```

```

selected_features = shap_feature_selection(opt.best_estimator_, X_train_final, top_n=1)
print("選出重要特徵:", selected_features)

# 用最佳參數重新訓練
final_model = XGBClassifier(**opt.best_params_, random_state=42, use_label_encoder=False, eval_metric='logloss')
final_model.fit(X_train_final[selected_features], y_train)

# 預測結果儲存
pred_test = final_model.predict_proba(X_test_final[selected_features])[:, 1]
result = pd.DataFrame({'id': test_id, 'smoking': pred_test})
result.to_csv("/content/drive/MyDrive/Data_Mining/final_result.csv", index=False)
print("📁 預測結果已儲存")

# 📊 Train vs Test 預測機率分佈比較
plt.figure(figsize=(10, 6))
plt.hist(final_model.predict_proba(X_train_final[selected_features])[:, 1], bins=50, alpha=0.6, label='Train')
plt.hist(pred_test, bins=50, alpha=0.6, label='Test')
plt.xlabel("Predicted Probability")
plt.ylabel("Frequency")
plt.title("Train vs Test Prediction Distribution")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# 計算訓練集預測結果
y_pred_train = final_model.predict(X_train_final[selected_features])
y_train_proba = final_model.predict_proba(X_train_final[selected_features])[:, 1]

final_auc = roc_auc_score(y_train, y_train_proba)
print(f"\n🎯 最終模型 AUC (Train): {final_auc:.4f}")

cm = confusion_matrix(y_train, y_pred_train)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix (Train)")
plt.tight_layout()
plt.show()

print("\n📋 Classification Report (Train):")
print(classification_report(y_train, y_pred_train))

fpr, tpr, _ = roc_curve(y_train, y_train_proba)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {final_auc:.4f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title("ROC Curve (Train)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# ✅ 放在底部執行主程式
main(X_train_final, X_test_final, y_train, test['id'])

```

```

# ----- 套件載入 -----
import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from tqdm import tqdm
from sklearn.model_selection import StratifiedKFold, cross_val_predict
from sklearn.metrics import roc_auc_score, roc_curve, confusion_matrix, ConfusionMatrixDisplay
from xgboost import XGBClassifier
from skopt import BayesSearchCV
from skopt.space import Real, Integer
from scipy.stats import skew
from sklearn.preprocessing import RobustScaler, LabelEncoder
import warnings
warnings.filterwarnings('ignore')

# ----- 載入資料 -----
train = pd.read_csv('/content/drive/MyDrive/Data_Mining/train.csv')
test = pd.read_csv('/content/drive/MyDrive/Data_Mining/test.csv')

# ----- 特徵工程: 視力、聽力欄位處理 -----
def create_extra_features(df):
    best_vision = np.minimum(df['eyesight(left)'], df['eyesight(right)'])
    worst_vision = np.maximum(df['eyesight(left)'], df['eyesight(right)'])
    df['eyesight(left)'] = best_vision
    df['eyesight(right)'] = worst_vision
    best_hearing = np.minimum(df['hearing(left)'], df['hearing(right)'])

```

```

        worst_hearing = np.maximum(df['hearing(left)'], df['hearing(right)'])
        df['hearing(left)'] = best_hearing - 1
        df['hearing(right)'] = worst_hearing - 1

create_extra_features(train)
create_extra_features(test)

# ----- 特徵工程：衍生欄位 -----
def add_derived_features(df):
    df['BMI'] = df['weight(kg)'] / (df['height(cm)'] / 100) ** 2
    df['BP_ratio'] = df['systolic'] / df['relaxation']
    df['liver_ratio'] = df['ALT'] / df['AST']
    df['cholesterol_ratio'] = df['LDL'] / df['HDL']

add_derived_features(train)
add_derived_features(test)

# ----- 分離目標與特徵 -----
X_train = train.drop(columns=['smoking', 'id'])
y_train = train['smoking']
X_test = test.drop(columns=['id'])

# ----- 頻率編碼 & 特徵分離 -----
to_ohe = ['hearing(left)', 'hearing(right)', 'dental caries']
X_train_cont = X_train.drop(columns=to_ohe + ['Urine protein'])
X_test_cont = X_test.drop(columns=to_ohe + ['Urine protein'])

def frequency_encode(train_col, test_col):
    freq_map = train_col.value_counts(normalize=True)
    return train_col.map(freq_map), test_col.map(freq_map).fillna(0)

X_train_cont['age_freq'], X_test_cont['age_freq'] = frequency_encode(X_train_cont['age'], X_test_cont['age'])

# ----- loglp 偏態處理 -----
numeric_cols = X_train_cont.select_dtypes(include=['float64', 'int64']).columns.drop(['age_freq'])
skewed_features = [col for col in numeric_cols if skew(X_train_cont[col]) > 1]

log_transformed_features = []
for col in skewed_features:
    if (X_train_cont[col] >= 0).all() and (X_test_cont[col] >= 0).all():
        X_train_cont[col] = np.log1p(X_train_cont[col])
        X_test_cont[col] = np.log1p(X_test_cont[col])
        log_transformed_features.append(col)

# ----- 標準化 -----
sc = RobustScaler()
X_train_scaled = pd.DataFrame(sc.fit_transform(X_train_cont.drop(columns=['age_freq'])), columns=X_train_cont.drop(columns=['age_freq']).columns)
X_test_scaled = pd.DataFrame(sc.transform(X_test_cont.drop(columns=['age_freq'])), columns=X_test_cont.drop(columns=['age_freq']).columns)
X_train_scaled['age_freq'] = X_train_cont['age_freq']
X_test_scaled['age_freq'] = X_test_cont['age_freq']

# ----- 類別編碼 -----
X_train_ohe = pd.get_dummies(X_train[to_ohe], columns=to_ohe)
X_test_ohe = pd.get_dummies(X_test[to_ohe], columns=to_ohe)

le = LabelEncoder()
X_train['Urine protein'] = le.fit_transform(X_train['Urine protein'])
X_test['Urine protein'] = le.transform(X_test['Urine protein'])

# ----- 合併所有特徵 -----
X_train_final = pd.concat([X_train_scaled, X_train_ohe, X_train[['Urine protein']], axis=1)
X_test_final = pd.concat([X_test_scaled, X_test_ohe, X_test[['Urine protein']], axis=1)

# ----- 確保特徵一致 -----
X_test_final = X_test_final.reindex(columns=X_train_final.columns, fill_value=0)

# tqdm callback
class TqdmCallback:
    def __init__(self, total_iter):
        self.pbar = tqdm(total=total_iter)

    def __call__(self, res):
        self.pbar.update(1)
        if self.pbar.n % 5 == 0:
            print(f"[{self.pbar.n}/{self.pbar.total}] 進度更新")

# Learning curve + ROC 曲線 (僅 AUC)
def plot_model_evaluation(opt, X_train, y_train, cv):
    auc_scores = opt.cv_results_['mean_test_score']

    # AUC Learning Curve
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(range(1, len(auc_scores) + 1), auc_scores, marker='o', label='AUC Score')
    plt.axhline(y=max(auc_scores), color='r', linestyle='--', label=f"Best AUC = {max(auc_scores):.4f}")
    plt.xlabel("Iteration")

```

```

plt.ylabel("Mean AUC (CV)")
plt.title("XGBoost AUC Learning Curve")
plt.legend()
plt.grid(True)

# ROC Curve
y_pred_proba_cv = cross_val_predict(opt.best_estimator_, X_train, y_train, cv=cv, method='predict_proba')[:, 1]
fpr, tpr, _ = roc_curve(y_train, y_pred_proba_cv)
auc_score = roc_auc_score(y_train, y_pred_proba_cv)

plt.subplot(1, 2, 2)
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {auc_score:.4f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title("XGBoost ROC Curve")
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

def run_xgb(X_train_final, X_test_final, y_train, test_id):
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    xgb_model = XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='logloss')

    xgb_space = {
        'learning_rate': Real(0.01, 0.3, prior='log-uniform'),
        'n_estimators': Integer(100, 500),
        'max_depth': Integer(3, 10),
        'subsample': Real(0.5, 1.0),
        'colsample_bytree': Real(0.5, 1.0),
        'min_child_weight': Integer(1, 10),
        'gamma': Real(0, 1.0),
        'reg_alpha': Real(0, 1.0),
        'reg_lambda': Real(0, 1.0),
        'scale_pos_weight': Real(1, 10)
    }

    print("🔵 開始 XGBoost Bayesian Optimization...")
    opt = BayesSearchCV(
        xgb_model,
        search_spaces=xgb_space,
        cv=cv,
        n_iter=70,
        scoring='roc_auc',
        n_jobs=-1,
        random_state=42,
        verbose=0
    )

    start = time.time()
    opt.fit(X_train_final, y_train, callback=TqdmCallback(70))
    print(f"✅ 搜尋完成, 耗時: {time.time() - start:.2f} 秒")
    print(f"最佳參數: {opt.best_params_}")
    print(f"最佳 AUC: {opt.best_score_:.4f}")

    # 評估圖表
    plot_model_evaluation(opt, X_train_final, y_train, cv)

    # 預測 + 混淆矩陣
    y_pred_train = opt.predict(X_train_final)
    y_train_proba = opt.predict_proba(X_train_final)[:, 1]
    final_auc = roc_auc_score(y_train, y_train_proba)
    print(f"\n🔴 最終模型 AUC (Train): {final_auc:.4f}")

    cm = confusion_matrix(y_train, y_pred_train)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm)
    disp.plot(cmap="Blues")
    plt.title("Confusion Matrix (Train)")
    plt.tight_layout()
    plt.show()

    # 預測測試集
    final_pred_test = opt.predict_proba(X_test_final)[:, 1]
    result = pd.DataFrame({'id': test_id, 'smoking': final_pred_test})
    result.to_csv("/content/drive/MyDrive/Data_Mining/xgb_final_result.csv", index=False)
    print(f"🟡 測試集預測已儲存為 xgb_final_result.csv")

# 呼叫主函數
run_xgb(X_train_final, X_test_final, y_train, test['id'])

```

```

-----
KeyError                                Traceback (most recent call last)
/usr/local/lib/python3.11/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3804         try:
-> 3805             return self._engine.get_loc(casted_key)
    3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'eyesight(left)'

```

The above exception was the direct cause of the following exception:

```

KeyError                                Traceback (most recent call last)
<ipython-input-5-176ef7388780> in <cell line: 0>()
     33
     34 create_extra_features(train)
--> 35 create_extra_features(test)
     36
     37 # ----- 特徵工程：衍生欄位 -----

<ipython-input-5-176ef7388780> in create_extra_features(df)
     23 # ----- 特徵工程：視力、聽力欄位處理 -----
     24 def create_extra_features(df):
--> 25     best_vision = np.minimum(df['eyesight(left)'], df['eyesight(right)'])
     26     worst_vision = np.maximum(df['eyesight(left)'], df['eyesight(right)'])
     27     df['eyesight(left)'] = best_vision

----- 1 frames -----
/usr/local/lib/python3.11/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
    3810         ):
    3811             raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
    3813         except TypeError:
    3814             # If we have a listlike key, _check_indexing_error will raise

KeyError: 'eyesight(left)'

```

```

class TqdmCallback:
    def __init__(self, total_iter):
        self.pbar = tqdm(total=total_iter)

    def __call__(self, res):
        self.pbar.update(1)
        if self.pbar.n % 5 == 0:
            print(f"[{self.pbar.n}/{self.pbar.total}] 進度更新")

def plot_model_evaluation(opt, X_train, y_train, cv, title_prefix="Model"):
    auc_scores = opt.cv_results_['mean_test_score']

    # AUC Learning Curve
    plt.figure(figsize=(14, 5))
    plt.subplot(1, 2, 1)
    plt.plot(range(1, len(auc_scores)+1), auc_scores, marker='o', label='AUC Score')
    plt.axhline(y=max(auc_scores), color='r', linestyle='--', label=f"Best AUC = {max(auc_scores):.4f}")
    plt.xlabel("Iteration")
    plt.ylabel("Mean AUC (CV)")
    plt.title(f"{title_prefix} AUC Learning Curve")
    plt.legend()
    plt.grid(True)

    # ROC Curve
    y_pred_proba_cv = cross_val_predict(opt.best_estimator_, X_train, y_train, cv=cv, method='predict_proba')[:, 1]
    fpr, tpr, _ = roc_curve(y_train, y_pred_proba_cv)
    auc_score = roc_auc_score(y_train, y_pred_proba_cv)

    plt.subplot(1, 2, 2)
    plt.plot(fpr, tpr, label=f"ROC Curve (AUC = {auc_score:.4f})")
    plt.plot([0, 1], [0, 1], 'k--')
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f"{title_prefix} ROC Curve")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

def shap_feature_selection(model, X_train, top_n=30):
    explainer = shap.TreeExplainer(model)
    shap_values = explainer.shap_values(X_train)

```

```

mean_abs_shap = np.abs(shap_values).mean(axis=0)
shap_importance = pd.DataFrame({
    'feature': X_train.columns,
    'importance': mean_abs_shap
}).sort_values(by='importance', ascending=False)

shap.summary_plot(shap_values, X_train, max_display=top_n, plot_type="bar")
return shap_importance.head(top_n)['feature'].tolist()

def main(X_train_final, X_test_final, y_train, test_id):
    # 定義模型與搜尋空間
    models = {
        "XGBoost": XGBClassifier(random_state=42, use_label_encoder=False, eval_metric='logloss'),
        "LightGBM": LGBMClassifier(random_state=42),
        "CatBoost": CatBoostClassifier(task_type="GPU", random_state=42, verbose=0)
    }

    param_spaces = {
        "XGBoost": {
            'learning_rate': Real(0.01, 0.3, prior='log-uniform'),
            'n_estimators': Integer(100, 500),
            'max_depth': Integer(3, 10),
            'subsample': Real(0.5, 1.0),
            'colsample_bytree': Real(0.5, 1.0),
            'min_child_weight': Integer(1, 10),
            'gamma': Real(0, 1.0),
            'reg_alpha': Real(0, 1.0),
            'reg_lambda': Real(0, 1.0),
            'scale_pos_weight': Real(1, 10)
        },
        "LightGBM": {
            'learning_rate': Real(0.01, 0.3, prior='log-uniform'),
            'n_estimators': Integer(100, 500),
            'max_depth': Integer(3, 10),
            'num_leaves': Integer(20, 100),
            'min_child_samples': Integer(5, 50)
        },
        "CatBoost": {
            'iterations': Integer(100, 1000),
            'depth': Integer(3, 10),
            'learning_rate': Real(0.01, 0.3, prior='log-uniform'),
            'l2_leaf_reg': Real(1, 10),
            'border_count': Integer(32, 255)
        }
    }

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

    results = {}
    for model_name, model in models.items():
        print(f"🚧 開始 {model_name} Bayesian Optimization...")
        opt = BayesSearchCV(
            model,
            search_spaces=param_spaces[model_name],
            cv=cv,
            n_iter=70,
            scoring='roc_auc',
            n_jobs=-1,
            random_state=42,
            verbose=0
        )

        start = time.time()
        opt.fit(X_train_final, y_train, callback=TqdmCallback(70))
        print(f"✅ 搜尋完成, 耗時: {time.time() - start:.2f} 秒")

        print(f"最佳參數: {opt.best_params_}")
        print(f"最佳 AUC: {opt.best_score_:.4f}")

        plot_model_evaluation(opt, X_train_final, y_train, cv, title_prefix=model_name)

        selected_features = shap_feature_selection(opt.best_estimator_, X_train_final, top_n=15)
        print(f"選出重要特徵: {selected_features}")

        results[model_name] = (opt.best_estimator_, selected_features)

    # Voting Classifier
    print(f"🗳 組合 Voting Classifier...")
    voting_clf = VotingClassifier(estimators=[(name, model) for name, (model, _) in results.items()], voting='soft')
    voting_clf.fit(X_train_final[[name for name, _ in results.items()]], y_train)

    # 預測結果
    final_pred_test = voting_clf.predict_proba(X_test_final[[name for name, _ in results.items()]])[:, 1]
    result = pd.DataFrame({'id': test_id, 'smoking': final_pred_test})
    result.to_csv("/content/drive/MyDrive/Data Mining/final_result.csv", index=False)
    print(f"📁 預測結果已儲存")

```



```

# 訓練集預測結果
y_pred_train = voting_clf.predict(X_train_final[[name for name, _ in results.items()]])
y_train_proba = voting_clf.predict_proba(X_train_final[[name for name, _ in results.items()]])[:, 1]
final_auc = roc_auc_score(y_train, y_train_proba)
print(f"\n🏆 最終模型 AUC (Train): {final_auc:.4f}")

cm = confusion_matrix(y_train, y_pred_train)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues")
plt.title("Confusion Matrix (Train)")
plt.tight_layout()
plt.show()

print("\n📊 Classification Report (Train):")
print(classification_report(y_train, y_pred_train))

fpr, tpr, _ = roc_curve(y_train, y_train_proba)
plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {final_auc:.4f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title("ROC Curve (Train)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

main(X_train_final, X_test_final, y_train, test['id'])

```